

Funzioni di Hash

$$h: U \rightarrow \{0, 1, \dots, m-1\}$$

$$\text{chiaro: } \{0, 1, 2, \dots, |U|-1\}$$

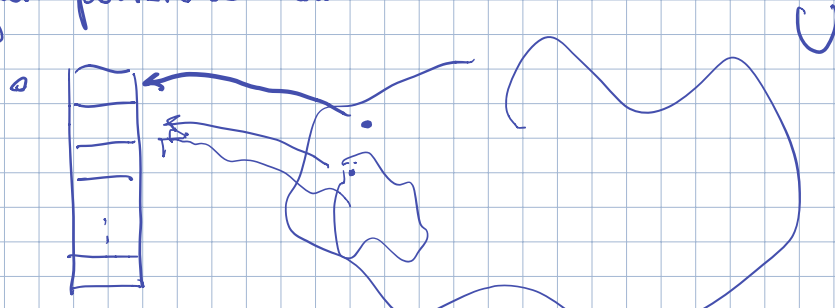
- Effettivamente $\forall x \in U \quad h(x.\text{key}) \in [0..m-1]$
- * $|U| \gg m$ sicuramente $\exists x, y$ t.c. $h(x.\text{key}) = h(y.\text{key})$
- $\forall j \in [0..m-1] \exists x$ t.c. $h(x.\text{key}) = j$

Per ogni posizione di T deve esistere almeno
un elemento di U che finirebbe in quella
posizione

Suriettiva

* h sicuramente non è iniettiva

- "Equiprobabilità" circa $\frac{|U|}{m}$ elementi di U
per ogni posizione di T



Ipotesi di hashing uniforme semplice

Funzioni di hash standard

Chiavi numeri interi

1) Metodo della divisione

$$h(k) = k \bmod m$$

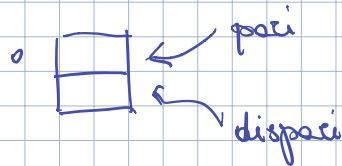
1, 2, ..., $|U|$

$|U| \gg m$ h è suriettiva

o 1, 2, ..., $|U|-1$

La distribuzione dipende da m

Esempio $m=2$ $\Leftarrow$ quando solo l'ultimo bit della chiave



$m = 2^p$ $\Leftarrow$ quando solo gli ultimi p bit

m m.zo primo Contano da potenze di 2

2) Metodo della moltiplicazione

Chiavi $0 \leq k < 1$

$$h(k) = \lfloor k \cdot m \rfloor \quad 0 \leq \lfloor k \cdot m \rfloor < m-1$$

Bucket Sort

A ha elem. unif. dist. in $[0, 1)$

$\Rightarrow$ k soddisfa ipotesi di hashing unif. semplice

B viene riempito in modo uniforme

$\Rightarrow$
Bucket Sort Lineare

Chiavi intere $0, 1, \dots, |U|-1$
 $1, 2, \dots, |U|$

Scelgo un numero R $0 < R < 1$

Data una chiave k

$$0 \leq k \cdot R - \lfloor k \cdot R \rfloor < 1$$

$$h(k) = \lfloor (k \cdot R - \lfloor k \cdot R \rfloor) \cdot m \rfloor$$

parte frazionaria $k \cdot R$

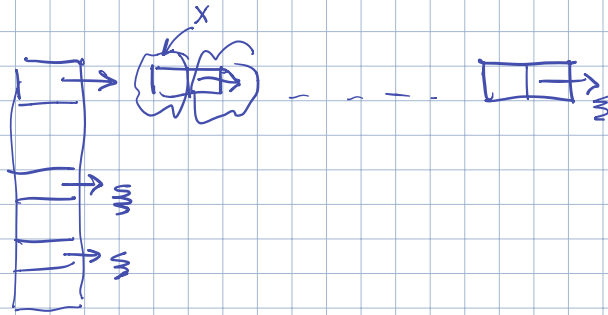
applico il met.
moltip.

non
devo
modificare
 m

HASH con Open Addressing

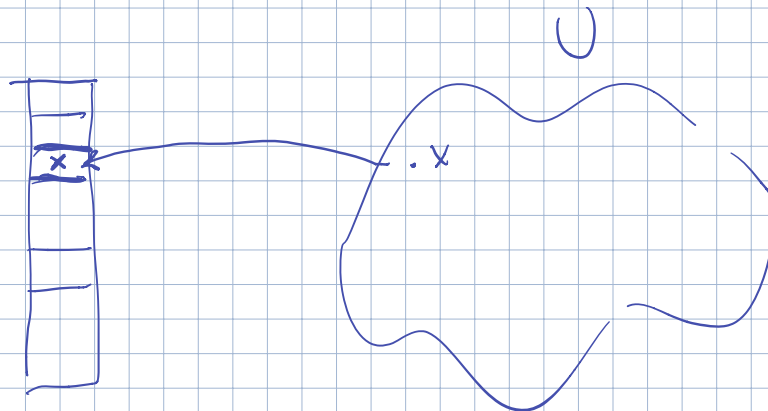
Hash con Chaining

Non so a priori quanto spazio occuperò



Hash con Open Addressing

- Fisso a priori lo spazio che dedicherò alla tabella
- Non utilizzo nessuna struttura esterna alla tabella

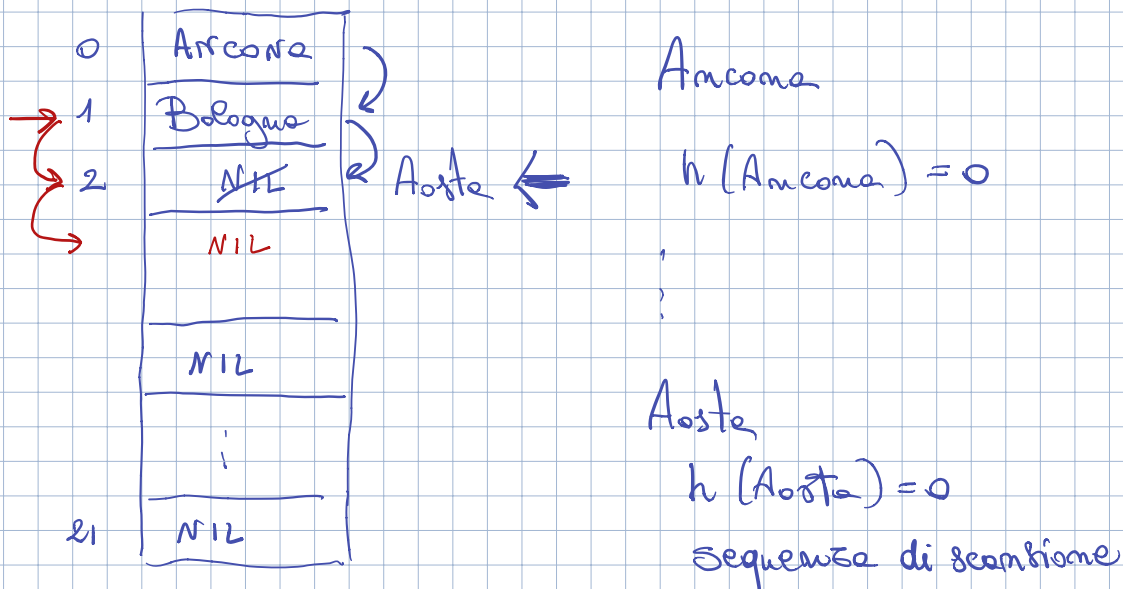


Dato x calcolo $h(x.key)$ e memorizzo
 x in $T[h(x.key)]$

Cosa faccio se è già occupata

Esempio

Nomi comuni $|T| = 21$



Funzione di Hash con sequenza di scartione

$$h : U \times \underbrace{\{0, 1, \dots, m-1\}}_{\text{tentativo}} \rightarrow \{0, 1, \dots, m-1\}$$

$$\forall k \quad \langle \underbrace{h(k,0), h(k,1), \dots, h(k,m-1)}_m \rangle$$

$\underbrace{\qquad\qquad\qquad}_{0 \dots m-1} \quad \underbrace{\qquad\qquad\qquad}_{0 \dots m-1} \quad \underbrace{\qquad\qquad\qquad}_{0 \dots m-1}$

Non deve avere ripetizioni $\Rightarrow$ È una perm. degli indici $0, 1, \dots, m-1$

Es.

$$|P| = 5$$

$$k = 100 \quad \langle 0, 3, 1, 4, 2 \rangle$$

$$k = 50 \quad \langle 0, \underset{\substack{\uparrow \\ m}}{3}, \underset{\substack{\uparrow \\ m}}{1}, \underset{\substack{\uparrow \\ m}}{4}, \underset{\substack{\uparrow \\ m}}{3} \rangle$$

Esempio

Se impongo che $\forall k$

$$\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$$

$\longrightarrow$

$$\langle 0, 1, 2, \dots, m-1 \rangle$$

No!

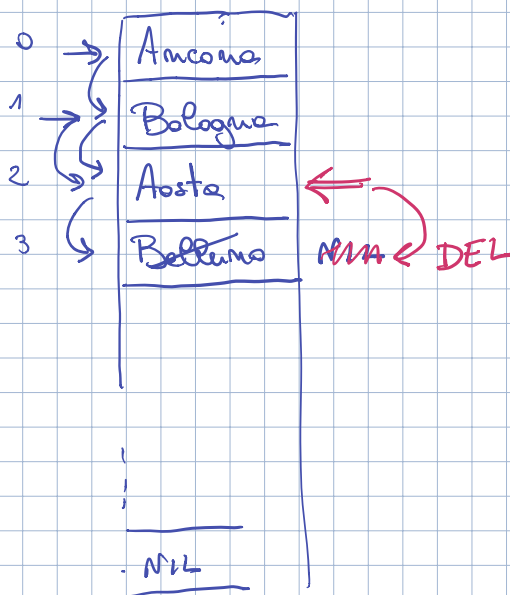
Se le sequenze sono

$$\langle h_0(k), h_0(k)+1, h_0(k)+2, \dots, m-1, 0, 1, \dots, h_0(k)-1 \rangle$$

Scansione lineare $\Rightarrow$ Primary Clustering

- 1) Come mi comporta con le celle vuote
- 2) Caratteristiche delle sequenze di scansione

1) Esempio



Inserito

Ancona
Bologna
Aosta
Belluno

Cancellato

Belluno

Cercato

Cagliari

$$h(\text{Cagliari}, 0) = 2$$

$$h(\text{Cagliari}, 1) = 3$$

Quando cancello scrivo DEL

DEL $\rightarrow$ è come NIL se devo inserire

DEL $\mapsto$ è come una cella occupata per la ricerca

La ricerca di x prosegue fino a che:

- trovo x oppure $\mapsto (x \text{ è in } T)$
- trovo NIL oppure $\mapsto (x \text{ non è in } T)$
- ho controllato $\langle h(x.\text{key}, 0), \dots, h(x.\text{key}, m-1) \rangle$
 $\mapsto (x \text{ non è in } T)$

Hash-Insert-Open (T, x, h) {

$m \leftarrow T.\text{length}$

$i \leftarrow 0$

$j \leftarrow h(x.\text{key}, i)$

while ($T[j] \neq \text{NIL} \text{ e } T[j] \neq \text{DEL} \text{ e } i < m$) {

$i \leftarrow i + 1$

if ($i = m$)

$\rightarrow j \leftarrow h(x.\text{key}, i)$ $\% \text{ controllare il caso limite } i = m$

}

if ($i = m$) {

return "non c'è posto"

} else {

$T[j] \leftarrow x$
return j

}

}

$\Theta(m)$

$\Theta(m)$ caso peggiore